

CCP4213 DATA STRUCTURE & ALGORITHMS

Bakery shop program

Lecturer's Name: Ruzanna Abdullah

October/November 2025 (Term 2530)

Prepared by TC1L Group 18

No.	Student ID	Student Name
1.	243DC243FZ	MUHAMMAD ARFAN BIN SHAHNAZ FARIZ
2.	243DC241D6	MUTAZ MOHAMED ABBAS SAAD
3.	243DC245LK	LEE ZHE HONG
4.	243DC245CV	ISABELLE S LOJUNTIN

Introduction

The Bakery shop program is a simple shop management program that help staff of the bakery to, Add, Edit, Delete, Display, And Search for items they are selling and allow them to take orders from that list of items as well as serve them using various systems as well as an external txt file to store data.

The objectives are to provide staff of the bakery with a fast easy system that manage their stock, orders, and serving said orders, various features and algorithms to keep the data secure and accessible.

The scope of the program is limited to the bakery's stock, customer orders, and items management.

The target user is bakery staff and management.

Table of Contents

Introduction.....	2
CRUD operations.....	3
Order creation	4
Order Management System.....	6
Item Management:	13
Data Structure and Algorithms.....	16
Hashing array with chain collision (hashing and linked list):.....	16
Searching (Hashed then Linear).....	16
Sorting (Bubble).....	17
Stack and Queue array implementation	18
Program output rundown	20

CRUD operations

There are various different systems used to provide consistent, accurate and precise results:

The system will be split into three sections:

- Order creation
- Order management
- Item management

Order creation

Made primarily by Mutaz Mohammad, order creation handles when a customer orders something from the menu

```
773 class order_stack{
774 private:
775     order_list stack; // stack container
776     int Head = -1; // top of stack
777     itemnode *temp; // pointer for traversal
778     double total_price = 0; // running total
779
780 public:
781
782     // push item into stack
783     void add_item(char letter, int number){
784
785         int index;
786         index = hashcode_char_to_int(letter); // convert letter to index
787
788         temp = &itemarray[index]; // go to hashed location
789
790         // move to correct item in linked list
791         for(int i=0;i<(number-1);i++){
792             if(temp!=NULL){
793                 temp = temp->link;
794             }
795         }
796
797         // check if item exists
798         if(temp!=NULL && temp->value.name!=""){
799             Head++; // move stack top
800
801             stack.name[Head] = temp->value.name;
802             stack.price[Head] = temp->value.cost;
803
804             total_price += temp->value.cost; // update total
805             stack.total_price = total_price;
806         }
807         else{
808             cout<<"item not found"<<endl;
809         }
810     }
811
812     // pop item from stack
813     void remove_item(){
814
815         if(Head == -1){
816             cout<<"stack is empty"<<endl;
817         }
818         else{
819             total_price -= stack.price[Head]; // subtract price
820             stack.name[Head] = "";
821             stack.price[Head] = 0;
822             Head--; // move top down
823         }
824     }
825
826     // display stack from top to bottom
827     void display_stack(){
828
829         for(int i = Head; i >= 0; i--){
830             cout<<"Name: "<<stack.name[i]
831                 <<"\t\tPrice: RM"<<stack.price[i]
832                 <<endl;
833         }
834     }
835
836     // return the final order
837     order_list confirm_order(){
838         order_id_generator(&stack);
839         return stack;
840     }
841 };
842
843 order_list order_menu() {
```

This part focuses in using a stack to organize customer orders within the bakery system. The order_list struct holds the names of the items, their prices, and the overall total, all stored in arrays with a fixed size to control the number of orders.

To manage the stack, the order_stack class is introduced with private variables to maintain data security.

The add_item function adds a selected item to the top of the stack after identifying it through the hash function, and the total price is updated accordingly.

The remove_item function removes the latest item added, ensuring that the total price reflects the change

```

843 order_list order_menu() {
844     order_stack current_order;
845     int choice;
846     char letter;
847     string input;
848     int number;
849
850     do{
851         decor_line();
852         cout<<"1. Add Item"<<endl;
853         cout<<"2. Remove Item"<<endl;
854         cout<<"3. Display Order"<<endl;
855         cout<<"4. Confirm Order"<<endl;
856         cout<<"5. Cancel"<<endl;
857         decor_line();
858         choice = get_int_input();
859
860         switch(choice){
861
862             case 1:
863                 display_details_alphabet();
864                 cout<<"Enter first letter of item: ";
865                 cin>>letter;
866
867                 cout<<"Enter item number: ";
868                 cin>>number;
869
870                 current_order.add_item(letter,number);
871                 break;
872
873             case 2:
874                 current_order.remove_item();
875                 break;
876
877             case 3:
878                 current_order.display_stack();
879                 break;
880
881             case 4:
882                 return current_order.confirm_order();
883
884             case 5:
885                 return order_list();
886             default:
887                 choice = 0;
888                 break;
889         }
890     }while(choice!=6);
891 }
892
893
894

```

The display_stack function outputs the current order starting from the top element downwards, showing the last in first out behavior that defines a stack.

The confirm_order function then returns the completed order once the user is satisfied.

The order_menu function acts as the control center of this section by repeatedly presenting options for adding, removing, viewing, or confirming items. add a few lines but dont change anything

Order Management System

The decor_2 and decor_3 functions are for titles and contents display stylization. decor_2 for displaying the menu name in between equal signs (=) and decor_3 for displaying the item name and price.

```
void decor_2(string x) // ex. ---- Options ----
{
    cout<<"==== "<<x<<"===="<<endl;
}

void decor_3(string x, double y) // ex. Lemon Cheesecake --- RM 18
{
    cout <<x<<" --- RM "<<y<<endl;
}
```

The order_list struct is for keeping the orders of each customer such as the order id, order item names, order item prices and the total price of the order(s).

```
struct order_list // Details for the orders for each instance
{
    string id;
    string name[order_limit];
    double price[order_limit];
    double total_price;
};
```

The queue struct is for creating the customer queue. A universal queue has been made called main_queue.

```
struct queue // The queue
{
    int head,tail;
    order_list orders[order_limit];
};
```

```
queue main_queue; // Main queue created during launch for immediate usage
```

The order_id_generator creates an identification number in the style of “Cxx” where C stands for customer and xx being numbers between 01-99. When the order ID reaches C99 it resets back to C01 for number reuse.

```
void order_id_generator(order_list *ref_order) // Generates the Order Identification Reference
{
    static int id_num1 = 1;
    static int id_num2 = 0;
    ref_order->id = "";
    ref_order->id += "C";
    ref_order->id += id_num2+48; // ASCII
    ref_order->id += id_num1+48; // ASCII
    id_num1++;
    if(id_num1>9)
    {
        id_num2++;
        id_num1=0;
        if (id_num2>9) // When the ID number reached 99 the next ID number will reset back to 01 for ID reuse
        {
            id_num2=0;
            id_num1++;
        }
    }
}
```

[illegible]

The queue_menu function prints out all order ID's within the system and calls for the options function.

```
void queue_menu(queue *main_queue) // The queue management menu filled
{
    decor_2("Order IDs");
    for (int i = main_queue->head; i <= main_queue->tail; i++)
    {
        cout<<main_queue->orders[i].id<<endl;
    }
    decor_line();
    options();
}
```

The options function offers the user options such as serve top order, view customer orders, cancel top order and back to Main Menu. A switch has been added to send the user to their respective choice. An error handling case has been added to prompt the user to try again.

```
void options() // The options for the queue management menu
{
    if(main_queue.head == main_queue.tail)
    {
        main_queue.head = 0;
        main_queue.tail = 0;
    }
    decor_2("Options");
    cout<<"[1] Serve top order\n[2] View customer orders\n[3] Cancel top order\n[4] Return to Main Menu"<<endl;
    decor_line();
    int choice;
    cout<<"Input Option Here : ";
    cin>>choice;
    decor_line();

    switch (choice)
    {
        case 1: // Serve top order
            serve_top(&main_queue);
            break;

        case 2: // View customer orders
            view_orders(&main_queue);
            break;

        case 3: // Cancel top order
            cancel_top(&main_queue);
            break;

        case 4: //Return back to the main menu
            return;

        default: // Error handling

            cout<<"Option not found! Please input a valid option."<<endl;
            return options();
            break;
    }

    return;
}
```

The `serve_top` function serves the order ID that is at the front of the queue by displaying the order id, the names of the ordered items and their respective prices, and the total price of the order. After that, the `main_menu->head` moves to the next ID in the queue before prompting the user to input another option.

```
void serve_top(queue *main_queue) // Serves the top order in the queue
{
    int i = 0;
    itemnode *temp;
    if(main_queue->head == main_queue->tail) // Checks if the queue is empty
    {
        cout<<"No orders available"<<endl;
        decor_line_2();
        return options();
    }
    else
    {
        main_queue->orders[main_queue->head].total_price = 0; // Initializing the total price

        decor_2("Now Serving " + main_queue->orders[main_queue->head].id); // Prints out order ID
        while (main_queue->orders[main_queue->head].name[i] != "" || !main_queue->orders[main_queue->head].name[i].empty()) // Makes sure the queue's order list is
        {
            decor_3(main_queue->orders[main_queue->head].name[i], main_queue->orders[main_queue->head].price[i]); // Prints out the order item's name and price
            temp = search_item_without_text(main_queue->orders[main_queue->head].name[i]); // Searches and minus the quantity of the item
            temp->value.quantity--;
            if(temp->value.quantity<0){temp->value.quantity=0;} // If quantity is below zero, it becomes 0 again because you can't have negative stock of items
            main_queue->orders[main_queue->head].total_price += main_queue->orders[main_queue->head].price[i]; // Adds up the total price
            i++;
        }
        cout<<endl;
        decor_2("Total Price");
        cout<<"RM "<<main_queue->orders[main_queue->head].total_price<<endl; // Prints out total price
        main_queue->head++; // Heads to the next order ID
    }

    decor_line_2();
    options(); // Prompt back the queue management menu
}
```

The `view_order` function showcases all available orders that have not been served and the items and price within each order ID. Once all orders have been displayed, the line “End of available orders” will be shown and the user will be prompted to input another option.

```
void view_orders(queue *main_queue) // View incompletd orders
{
    for (int i = main_queue->head; i < main_queue->tail; i++) // Encompasses all incompletd orders
    {
        int j = 0;
        decor_2(main_queue->orders[i].id);
        while (main_queue->orders[i].name[j] != "" || !main_queue->orders[i].name[j].empty()) // Makes sure
        {
            decor_3(main_queue->orders[i].name[j], main_queue->orders[i].price[j]); // Prints out the order
            j++;
        }
        decor_line();
    }
    cout<<"End of available orders"<<endl;
    decor_line_2();
    options(); // Prompt back the queue management menu
}
```

The `cancel_top` function cancels the serving option for the order ID that is at the front by displaying the order ID and order details of the order being cancelled before having the `main_queue->head` go to the next order in line.

```
void cancel_top(queue *main_queue) // To search and delete a spesific order
{
    int i = 0;

    if(main_queue->head == main_queue->tail) // Checks if the queue is empty
    {
        cout<<"No orders available"<<endl;
        decor_line_2();
        return options();
    }
    else
    {
        main_queue->orders[main_queue->head].total_price = 0; // Initializing the total price

        decor_2("Cancelling Order " + main_queue->orders[main_queue->head].id); // Prints out order ID
        while (main_queue->orders[main_queue->head].name[i] != "" || !main_queue->orders[main_queue->head].name[i].empty()) // Makes
        {
            decor_3(main_queue->orders[main_queue->head].name[i], main_queue->orders[main_queue->head].price[i]); // Prints out the o
            main_queue->orders[main_queue->head].total_price += main_queue->orders[main_queue->head].price[i]; // Adds up the total p
            i++;
        }
        cout<<endl;
        decor_2("Total Price");
        cout<<"RM "<<main_queue->orders[main_queue->head].total_price<<endl; // Prints out total price
        main_queue->head++; // Heads to the next order ID
    }

    decor_line_2();
    options(); // Prompt back the queue management menu
}
```

Item Management:

Item Management handles direct management with the items, this includes, adding, editing, displaying, searching and deleting items.

```
// items refer to a base value for all Items
struct item{
    string name;
    double cost;
    int quantity;
    item(){
        name = "";
        cost = 0;
        quantity = 0;
    }
    item(string n, double p, int q){
        name = n;
        cost = p;
        quantity = q;
    }
};

//itemnode is a specific linked list structure for items
struct itemnode{
    item value;
    itemnode *link;
    //LEE ZHE HONG addition
    itemnode(){
        value.name = "";
        value.cost = 0.0;
        link = NULL;
    }

    itemnode(string n, double p){
        value.name = n;
        value.cost = p;
        link = NULL;
    }
    //LEE ZHE HONG end of addition
};

itemarray[26]; // the Global Array of itemnode, in the range of 26 for the letters in the alphabet
// this allows for a hash array with a chain collision operation, using linked lists to store more data
```

This is the base data types used, that being items and itemnode, to allow for linked trees, the global itemarray is declared here.

```
/*The hashcode_char_to_int function allows a char character to be accurately given a hash value based on the english Alphabet.
it is not case nensitive*/
int hashcode_char_to_int(char name){
    int converted_name = int(name);
    if(converted_name>65 and converted_name<90){
        converted_name+=32;
    }
    if(converted_name>97 and converted_name<122){
        converted_name-=97;
    }
    int output;
    output = converted_name % 26;
    return output;
}
```

The hashing function here allows the names to be used to assign a hash index to the itemnode. Which then is used to

store the item into that index in itemarray, if a collision occurs, a chain approach is used to make it into a linked tree and allow it to expend infinitely.

```
itemnode* search_item(string name){
    itemnode* temp;
    int index, count = 1;

    index = hashcode_char_to_int(name[0]);
    temp = &itemarray[index];

    while(temp->value.name != name && temp->link != NULL && temp->value.name != ""){
        count++;
        temp = temp->link;
    }

    if(temp->link != NULL){
        cout<<"Item found at index : ("<<index<<") and the ("<<count<<")nd chain"<<endl;
        return temp;
    } else{
        cout<<"Error : Item '"<<name<<"' was not found in the hash table."<<endl;
        return NULL;
    }
}
```

The search function searching through the itemarray for a matching name that was inputted. It first uses the first letter and find it's hash value, after words, it uses linear search to find the item with the name. if the item is found it returns the

item address, if not it returns NULL.

```

void delete_item(string name){
    itemnode *temp;
    itemnode *priv;
    int index, count = 1;

    index = hashcode_char_to_int(name[0]);

    temp = &itemarray[index];
    priv = temp;

    while(temp->value.name != name && temp->link != NULL && temp->value.name != ""){
        count++;
        priv = temp;
        temp = temp->link;
    }
    if(temp->link != NULL){
        cout<<"Item found at index : ("<<index<<") and the ("<<count<<")nd chain"<<endl;
        priv->link = temp->link;
        temp->link = NULL;

        if(temp != &itemarray[index]){
            cout<<"Item '"<<temp->value.name<<"' has been deleted successfully."<<endl;
            delete temp;
        }else{
            cout<<"Item '"<<temp->value.name<<"' has been deleted successfully."<<endl;
            temp->value.name = "";
            temp->value.cost = 0.0;
        }
    } else{
        cout<<"Error : Item '"<<name<<"' was not found in the hash table."<<endl;
    }
    return;
}

```

The delete_item function also uses the same searching algorithm as search to find the item, once it is found, the links are moved around so that items past this item is connected to previous items to safely delete this item from the linked list.

The Item management system offers different display to show the itemarray sorted differently,

```

//displays all values in the itemarray in Alphabetical order
void display_details_alphabet(){
    itemnode *temp;
    string char_converted;
    int count = 1;
}

```

```

void sorted_display_cost(){
    item* sortarr;
    int size = 1, count = 0;
    itemnode* temp;
}

```

```

void sorted_display_quantity(){
    item* sortarr;
    int size = 1, count = 0;
    itemnode* temp;
}

```

```

[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : 1
=====
==== Items Sorted By Name (Alphabetical Order) =====
==== A ====
[1] Item : Applepie_Slice | Cost : RM 4 | Quantity : 13
[2] Item : Applepie | Cost : RM 15 | Quantity : 2
=====
==== C ====
[1] Item : Chocolete_Muffin | Cost : RM 5 | Quantity : 7
[2] Item : Cake | Cost : RM 12 | Quantity : 5
=====
==== D ====
[1] Item : Donut | Cost : RM 3.5 | Quantity : 10
=====
==== L ====
[1] Item : Lemon_Cheesecake | Cost : RM 16 | Quantity : 5
=====
==== M ====
[1] Item : Muffin | Cost : RM 5 | Quantity : 7
=====
==== S ====
[1] Item : Strawberry_Donut | Cost : RM 4 | Quantity : 8
=====
==== V ====
[1] Item : Vanilla_Muffin | Cost : RM 5 | Quantity : 8
=====

```

```

[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : 2
=====
==== Items Sorted by Cost (lowest to Highest) =====
Item: Donut | Cost: RM 3.5 | Quantity: 10
Item: Applepie_Slice | Cost: RM 4 | Quantity: 13
Item: Strawberry_Donut | Cost: RM 4 | Quantity: 8
Item: Chocolete_Muffin | Cost: RM 5 | Quantity: 7
Item: Muffin | Cost: RM 5 | Quantity: 7
Item: Vanilla_Muffin | Cost: RM 5 | Quantity: 8
Item: Cake | Cost: RM 12 | Quantity: 5
Item: Applepie | Cost: RM 15 | Quantity: 2
Item: Lemon_Cheesecake | Cost: RM 16 | Quantity: 5
=====

```

```

[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : 3
=====
==== Items Sorted by Quantity (lowest to Highest) =====
Item: Applepie | Cost: RM 15 | Quantity: 2
Item: Cake | Cost: RM 12 | Quantity: 5
Item: Lemon_Cheesecake | Cost: RM 16 | Quantity: 5
Item: Chocolete_Muffin | Cost: RM 5 | Quantity: 7
Item: Muffin | Cost: RM 5 | Quantity: 7
Item: Strawberry_Donut | Cost: RM 4 | Quantity: 8
Item: Vanilla_Muffin | Cost: RM 5 | Quantity: 8
Item: Donut | Cost: RM 3.5 | Quantity: 10
Item: Applepie_Slice | Cost: RM 4 | Quantity: 13
=====

```

```

void add_item(string n,double p,int q){
    item temp_item(n,p,q);
    int index = hashcode_char_to_int(n[0]);
    itemnode *temp,*priv;
    temp = &itemarray[index]; //traversal of the chain
    while(temp!=NULL && temp->value.name != temp_item.name){
        priv = temp;
        temp = temp->link;
    }
    if(temp==NULL){
        cout<<"Item added successfully";
        temp = new itemnode; //creates an empty new item for the next item to be added
        temp->link=NULL;
        temp->value.name = "";
        temp->value.cost = 0.0;
        temp->value.quantity = 0;
        priv->link = temp;
        priv->value = temp_item;
    }
    else{
        cout<<"Item is already in the itemarray";
    }
    cout<<endl;
}

```

The add_item function allows the user to add items into the itemarray.

This switch case allows the user to search and edit a specific item in the itemarray.

```

case 5://searches and edits a specific item in the itemarray
    temp_name = "";
    cout<<"Enter the Item name(no spaces):";
    cin>>temp_name;
    cin.clear();
    temp_node = search_item(temp_name);
    choice = 0;
    while(choice!=4){
        decor_line();
        cout<<"Name: "<<temp_node->value.name<<"\n|Cost: RM"<<temp_node->value.cost<<"\n|Stock: "<<temp_node->value.quantity<<"|";
        decor_2("Options");
        cout<<"[1] Change name:\n";
        cout<<"[2] Change cost:\n";
        cout<<"[3] Change quiantity\n";
        cout<<"[4] Return:\n";
        choice = get_int_input();
    }
}

```

That is all CRUD in Item Management.

Data Structure and Algorithms

The Bakery shop program has many data structures and algorithms included, specifics will be shown below.

Hashing array with chain collision (hashing and linked list):

Using an array and linked lists, we are able to create an array that correlates to the English alphabet, with a array size of 26, the hash code is generated by first converting the first letter of a string input (such as item name) into an interger, this is done using ascii, the uppercase letters values are converted into lowercase letter values before subtracting the value by 97 (ascii code for lowercase “a” is 97) and then returns the remainder from division of 26. If a non-letter is the first, it will still be hashed, but only using the remainder from division of 26.

```
int hashcode_char_to_int(char name){
    int converted_name = int(name);
    if(converted_name>=65 and converted_name<=90){
        converted_name+=32;
    }
    if(converted_name>=97 and converted_name<=122){
        converted_name-=97;
    }
    int output;
    output = converted_name % 26;
    return output;
}

//Itemnode is a specific linked list structure for items
struct itemnode{
    item value;
    itemnode *link;
    //LEE ZHE HONG addition
    itemnode(){
        value.name = "";
        value.cost = 0.0;
        link = NULL;
    }
    itemnode(string n, double p){
        value.name = n;
        value.cost = p;
        link = NULL;
    }
    //LEE ZHE HONG end of addition
}
itemarray[26]; // the Global Array of itemnode, in the range of 26 for the letters in the alphabet
// this allows for a hash array with a chain collision operation, using linked lists to store more data
```

Searching (Hashed then Linear)

The searching algorithm we use the hashing algorithm to locate the letter group inside the itemarray, then it uses a Linear search algorithm to find the exact name requested.

```
itemnode* search_item(string name){
    itemnode* temp;
    int index, count = 1;

    index = hashcode_char_to_int(name[0]);
    temp = &itemarray[index];

    while(temp->value.name != name && temp->link != NULL && temp->value.name != ""){
        count++;
        temp = temp->link;
    }

    if(temp->link != NULL){
        cout<<"Item found at index : ("<<index<<") and the ("<<count<<")nd chain"<<endl;
        return temp;
    } else{
        cout<<"Error : Item '"<<name<<"' was not found in the hash table."<<endl;
        return NULL;
    }
}
```

Sorting (Bubble)

Sorting is used in our system to allow the user to view the items in the itemarray through 3 differently sorted displays, the sort by cost and sort by quantity first grabs all the items in the dynamic hashed array into a one dimensional array, and then uses bubble sort to sort the items before displaying it, for display name alphabetically, the itemarray is already sorted to be alphabetical.

```
void sorted_display_cost(){
    item* sortarr;
    int size = 1, count = 0;
    itemnode* temp;

    for(int i = 0; i < 26; i++){
        temp = &itemarray[i];
        while(temp != NULL){
            if(temp->value.name != ""){
                size++;
            }
            temp = temp->link;
        }
    }

    sortarr = new item[size];

    for(int i = 0; i < 26; i++){
        temp = &itemarray[i];
        while(temp != NULL){
            if(temp->value.name != ""){
                sortarr[count] = temp->value;
                count++;
            }
            temp = temp->link;
        }
    }

    for(int i = 0; i < count - 1; i++){
        for(int j = 0; j < count - i - 1; j++){
            if(sortarr[j].cost > sortarr[j + 1].cost){
                item tempItem = sortarr[j];
                sortarr[j] = sortarr[j + 1];
                sortarr[j + 1] = tempItem;
            }
        }
    }

    decor_2("Items Sorted by Cost (lowest to Highest)");
    decor_line();

    for(int i = 0; i < count; i++){
        cout<<"Item: "<<sortarr[i].name<<" | Cost: RM "<< sortarr[i].cost <<" | Quantity: "<<sortarr[i].quantity<<endl;
        decor_line();
    }
    decor_line_2();
    delete[] sortarr;
}
```

Stack and Queue array implementation

The stack implementation is used when punching in orders from the customer, the orders will be added and removed by a last in first out method.

```
Input Option Here : 3
Name: Strawberry_Donut | Price: RM4
Name: Applepie | Price: RM15
Name: Applepie_Slice | Price: RM4
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 2
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 3
Name: Applepie | Price: RM15
Name: Applepie_Slice | Price: RM4
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : █

[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
-----
==== Items Sorted By Name (Aphabetetical Order) ====
==== A ====
[1] Item : Applepie_Slice | Cost : RM 4 | Quantity : 13
[2] Item : Applepie | Cost : RM 15 | Quantity : 2
-----
==== C ====
[1] Item : Chocolate_Muffin | Cost : RM 5 | Quantity : 7
[2] Item : Cake | Cost : RM 12 | Quantity : 5
-----
==== D ====
[1] Item : Donut | Cost : RM 3.5 | Quantity : 10
-----
==== L ====
[1] Item : Lemon_Cheesecake | Cost : RM 16 | Quantity : 5
-----
==== M ====
[1] Item : Muffin | Cost : RM 5 | Quantity : 7
-----
==== S ====
[1] Item : Strawberry_Donut | Cost : RM 4 | Quantity : 8
-----
==== V ====
[1] Item : Vanilla_Muffin | Cost : RM 5 | Quantity : 8
-----
Enter first letter of item: M
Enter item number: 1
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 3
Name: Muffin | Price: RM5
Name: Applepie | Price: RM15
Name: Applepie_Slice | Price: RM4
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : █
```

The Queue implementation is in managing the orders that come from the customer. We can serve and cancel orders, as well as view specifics

```

===== Order IDs =====
C01
C02
-----
===== Options =====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 2
-----
===== C01 =====
Applepie_Slice --- RM 4
Applepie --- RM 15
Muffin --- RM 5
-----
===== C02 =====
Vanilla_Muffin --- RM 5
Chocolete_Muffin --- RM 5
Strawberry_Donut --- RM 4
-----
End of available orders
=====
===== Options =====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : █

-----
Input Option Here : 1
-----
===== Now Serving C01 =====
Applepie_Slice --- RM 4
Applepie --- RM 15
Muffin --- RM 5
-----
===== Total Price =====
RM 24
=====
===== Options =====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 2
-----
===== C02 =====
Vanilla_Muffin --- RM 5
Chocolete_Muffin --- RM 5
Strawberry_Donut --- RM 4
-----
End of available orders
=====
===== Options =====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : █

```

Program output rundown

Startup:

```
-----  
Bakery System  
-----  
|-----|Loading Complete  
Applepie_Slice  
Applepie  
Chocolete_Muffin  
Donut  
Lemon_Cheesecake  
Muffin  
Peanuts  
Strawberry_Donut  
Vanilla_Cake  
Vanilla_Muffin  
-  
-  
-  
-  
-  
==== Main Menu ====  
[1] Create customer order  
[2] Manage orders  
[3] Manage items  
[4] Exit system  
-----  
Input Option Here : █
```

```
=====
[1] Item : Vanilla_Cake | Cost : RM 13 | Quantity : 5
-----
[2] Item : Vanilla_Muffin | Cost : RM 5 | Quantity : 8
-----
Enter first letter of item: A 1
Enter item number:
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
-----
==== Items Sorted By Name (Alphabetical Order) ====
==== A ====
[1] Item : Applepie_Slice | Cost : RM 4 | Quantity : 12
-----
[2] Item : Applepie | Cost : RM 15 | Quantity : 1
-----
==== C ====
[1] Item : Chocolate_Muffin | Cost : RM 5 | Quantity : 6
-----
==== D ====
[1] Item : Donut | Cost : RM 3.5 | Quantity : 10
-----
==== L ====
[1] Item : Lemon_Cheesecake | Cost : RM 16 | Quantity : 5
-----
==== M ====
[1] Item : Muffin | Cost : RM 5 | Quantity : 7
-----
==== P ====
[1] Item : Peanuts | Cost : RM 3 | Quantity : 25
-----
==== S ====
[1] Item : Strawberry_Donut | Cost : RM 4 | Quantity : 8
-----
==== V ====
[1] Item : Vanilla_Cake | Cost : RM 13 | Quantity : 5
-----
[2] Item : Vanilla_Muffin | Cost : RM 5 | Quantity : 8
-----
Enter first letter of item: S 1
Enter item number:
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : p 1
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
-----
==== Items Sorted By Name (Alphabetical Order) ====
==== A ====
[1] Item : Applepie_Slice | Cost : RM 4 | Quantity : 12
-----
[2] Item : Applepie | Cost : RM 15 | Quantity : 1
-----
==== C ====
[1] Item : Chocolate_Muffin | Cost : RM 5 | Quantity : 6
-----
==== D ====
[1] Item : Donut | Cost : RM 3.5 | Quantity : 10
-----
==== L ====
[1] Item : Lemon_Cheesecake | Cost : RM 16 | Quantity : 5
-----
==== M ====
[1] Item : Muffin | Cost : RM 5 | Quantity : 7
-----
==== P ====
[1] Item : Peanuts | Cost : RM 3 | Quantity : 25
-----
==== S ====
[1] Item : Strawberry_Donut | Cost : RM 4 | Quantity : 8
-----
==== V ====
[1] Item : Vanilla_Cake | Cost : RM 13 | Quantity : 5
-----
[2] Item : Vanilla_Muffin | Cost : RM 5 | Quantity : 8
-----
Enter first letter of item: V 2
Enter item number:
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
-----
Name: Vanilla_Muffin | Price: RM3
Name: Strawberry_Donut | Price: RM4
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
-----
==== Items Sorted By Name (Alphabetical Order) ====
==== A ====
[1] Item : Applepie_Slice | Cost : RM 4 | Quantity : 12
-----
[2] Item : Applepie | Cost : RM 15 | Quantity : 1
-----
==== C ====
[1] Item : Chocolate_Muffin | Cost : RM 5 | Quantity : 6
-----
==== D ====
[1] Item : Donut | Cost : RM 3.5 | Quantity : 10
-----
==== L ====
[1] Item : Lemon_Cheesecake | Cost : RM 16 | Quantity : 5
-----
==== M ====
[1] Item : Muffin | Cost : RM 5 | Quantity : 7
-----
==== P ====
[1] Item : Peanuts | Cost : RM 3 | Quantity : 25
-----
==== S ====
[1] Item : Strawberry_Donut | Cost : RM 4 | Quantity : 8
-----
==== V ====
[1] Item : Vanilla_Cake | Cost : RM 13 | Quantity : 5
-----
[2] Item : Vanilla_Muffin | Cost : RM 5 | Quantity : 8
-----
Enter first letter of item: V 2
Enter item number:
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
-----
Name: Vanilla_Muffin | Price: RM5
Name: Vanilla_Cake | Price: RM13
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 1
```

view all orders in the manage orders menu

```
-----
[1] Add Order Item
[2] Remove Order Item
[3] Display Order Items
[4] Confirm Order
[5] Return to Main Menu
-----
Input Option Here : 4
==== Main Menu ====
[1] Create customer order
[2] Manage orders
[3] Manage items
[4] Exit system
-----
Input Option Here : 2
==== Order IDs ====
C01
C02
C03
-----
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 2
-----
==== C01 ====
ApplePie_Slice --- RM 4
Chocolete_Muffin --- RM 5
Vanilla_Muffin --- RM 5
-----
==== C02 ====
Strawberry_Donut --- RM 4
Peanuts --- RM 3
-----
==== C03 ====
Vanilla_Cake --- RM 13
Vanilla_Muffin --- RM 5
-----
End of available orders
=====
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : █
```

Serve 1 and cancel 1

check items sorted by quantity

```
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 1
-----
==== Now Serving C01 ====
Applepie_Slice --- RM 4
Chocolete_Muffin --- RM 5
Vanilla_Muffin --- RM 5

==== Total Price ====
RM 14
=====
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 3
-----
==== Cancelling Order C02 ====
Strawberry_Donut --- RM 4
Peanuts --- RM 3

==== Total Price ====
RM 7
=====
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 2
-----
==== C03 ====
Vanilla_Cake --- RM 13
Vanilla_Muffin --- RM 5
-----
End of available orders
=====
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : █

==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 4
-----
==== Main Menu ====
[1] Create customer order
[2] Manage orders
[3] Manage items
[4] Exit system
-----
Input Option Here : 3
-----
[1] Add Item
[2] Delete Item
[3] View Items
[4] Search Item
[5] Modifiy Item
[6] Add Item Stock
[7] Save
[8] Return
-----
Input Option Here : 3
[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : 3
==== Items Sorted by Quantity (lowest to Highest) ====
-----
Item: Applepie | Cost: RM 15 | Quantity: 1
-----
Item: Chocolete_Muffin | Cost: RM 5 | Quantity: 5
-----
Item: Lemon_Cheesecake | Cost: RM 16 | Quantity: 5
-----
Item: Vanilla_Cake | Cost: RM 13 | Quantity: 5
-----
Item: Muffin | Cost: RM 5 | Quantity: 7
-----
Item: Vanilla_Muffin | Cost: RM 5 | Quantity: 7
-----
Item: Strawberry_Donut | Cost: RM 4 | Quantity: 8
-----
Item: Donut | Cost: RM 3.5 | Quantity: 10
-----
Item: Applepie_Slice | Cost: RM 4 | Quantity: 11
-----
Item: Peanuts | Cost: RM 3 | Quantity: 25
-----
[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : █
```

Take note that when an order is served, items in that order gets their quantity lowered

Serve last and check by quantity again

```
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 1
-----
==== Now Serving C03 ====
Vanilla_Cake --- RM 13
Vanilla_Muffin --- RM 5

==== Total Price ====
RM 18
=====
==== Options ====
[1] Serve top order
[2] View customer orders
[3] Cancel top order
[4] Return to Main Menu
-----
Input Option Here : 4
-----
==== Main Menu ====
[1] Create customer order
[2] Manage orders
[3] Manage items
[4] Exit system
-----
Input Option Here : 3
-----
[1] Add Item
[2] Delete Item
[3] View Items
[4] Search Item
[5] Modifiy Item
[6] Add Item Stock
[7] Save
[8] Return
Input Option Here : 3
[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : 3
==== Items Sorted by Quantity (lowest to Highest) ====
-----
Item: Applepie | Cost: RM 15 | Quantity: 1
-----
Item: Vanilla_Cake | Cost: RM 13 | Quantity: 4
-----
Item: Chocolete_Muffin | Cost: RM 5 | Quantity: 5
-----
Item: Lemon_Cheesecake | Cost: RM 16 | Quantity: 5
-----
Item: Vanilla_Muffin | Cost: RM 5 | Quantity: 6
-----
Item: Muffin | Cost: RM 5 | Quantity: 7
-----
Item: Strawberry_Donut | Cost: RM 4 | Quantity: 8
-----
Item: Donut | Cost: RM 3.5 | Quantity: 10
-----
Item: Applepie_Slice | Cost: RM 4 | Quantity: 11
-----
Item: Peanuts | Cost: RM 3 | Quantity: 25
-----
=====
[1] Display alphabetical order
[2] Display sorted by cost
[3] Display sorted by quantity
[4] Return
:Input Option Here : █
```

Order that was served

```
==== Now Serving C03 ====
Vanilla_Cake --- RM 13
Vanilla_Muffin --- RM 5

==== Total Price ====
RM 18
```

old stock:

```
-----
Item: Vanilla_Cake | Cost: RM 13 | Quantity: 5
-----
```

```
-----
Item: Vanilla_Muffin | Cost: RM 5 | Quantity: 7
-----
```

New stock:

```
-----
Item: Vanilla_Cake | Cost: RM 13 | Quantity: 4
-----
```

```
-----
Item: Vanilla_Muffin | Cost: RM 5 | Quantity: 6
-----
```

Modify the values of Vanilla_Muffin

```
[1] Add Item
[2] Delete Item
[3] View Items
[4] Search Item
[5] Modifiy Item
[6] Add Item Stock
[7] Save
[8] Return
Input Option Here : 5
Enter the Item name(no spaces):Vanilla_Muffin
Item found at index : (21) and the (2)nd chain
-----
|Name: Vanilla_Muffin|
|Cost: RM5|
|Stock: 6|==== Options ====
[1] Change name:
[2] Change cost:
[3] Change quiantity
[4] Return:
:Input Option Here : 1
==== Input new item name ====

:Super_Vanilla_Muffin
-----
|Name: Super_Vanilla_Muffin|
|Cost: RM5|
|Stock: 6|==== Options ====
[1] Change name:
[2] Change cost:
[3] Change quiantity
[4] Return:
:Input Option Here : 2
==== Input new item cost ====

:7
-----
|Name: Super_Vanilla_Muffin|
|Cost: RM7|
|Stock: 6|==== Options ====
[1] Change name:
[2] Change cost:
[3] Change quiantity
[4] Return:
:Input Option Here : 3
==== Input new item quantity ====

:8
-----
|Name: Super_Vanilla_Muffin|
|Cost: RM7|
|Stock: 8|==== Options ====
[1] Change name:
[2] Change cost:
[3] Change quiantity
[4] Return:
:Input Option Here : 4
-----
[1] Add Item
[2] Delete Item
[3] View Items
[4] Search Item
[5] Modifiy Item
[6] Add Item Stock
[7] Save
[8] Return
Input Option Here : 
```

Exit and save

```
-----
[1] Add Item
[2] Delete Item
[3] View Items
[4] Search Item
[5] Modifiy Item
[6] Add Item Stock
[7] Save
[8] Return
Input Option Here : 8
==== Main Menu ====
[1] Create customer order
[2] Manage orders
[3] Manage items
[4] Exit system
-----
Input Option Here : 4
Save?[y/n] : y
Saved succesfully
-----
Thank you for using the bakery system
Programme terminated
=====
```

New items_data

```
1  Applepie_Slice 4 11
2  Applepie 15 1
3  Chocolete_Muffin 5 5
4  Donut 3.5 10
5  Lemon_Cheesecake 16 5
6  Muffin 5 7
7  Peanuts 3 25
8  Strawberry_Donut 4 8
9  Vanilla_Cake 13 4
10 Super_Vanilla_Muffin 7 8
11
```